《前端框架专用周》

报 告 书

系　　部：物联网工程学院

专　　业：软件技术

课题名称：蜜雪冰城点单系统

班　　级：（已脱敏）

学　　号：（已脱敏）

姓　　名：谢凡

指导教师：（已脱敏）（已脱敏）（已脱敏）

无锡职业技术学院物联网工程学院

软件技术系

2025 年 4 月 30 日

目录

一、系统分析.....2
1、系统概述.....2
2、开发平台及工具选择.....2
二、功能分析.....3
1、功能模块划分.....3
2、系统功能模块图.....4
三、uni-app 程序项目架构图.....4
四、程序的制作风格介绍.....5
五、系统实施.....6
1、首页.....6
2．店铺.....8
3．收货地址.....13
4．点餐.....19
5．订单.....35
6．我的.....39
6．登录与注册.....42
六、网站使用与维护.....48
1、使用说明.....48
2、维护方案.....48
专用周小结.....49
参考文献.....51

《前端框架专用周》任务书

系部 软件技术系 班级 （已脱敏） 学号 （已脱敏） 姓名 谢凡

一、题目：蜜雪冰城点单系统

二、目的要求

1. 培养学生网页设计和排版的能力，使学生具备各类主题网页设计的能力，了解网页设计与制作原理，并能通过观摩优秀作品，提高学生审美能力。

2. 训练学生在遵守纪律、掌握进度、思维独立、群体协作等非认知因素方面具备能力。

三、内容

1. 确定课题，进行项目描述与项目设计

基于 uni-app 技术设计一个具有一定主题的移动端网站，网站包括首页、内容页、导航、广告展示等。

2. 项目实施

编码实现项目，在项目实施过程中，要逐步形成进度推进和质量保证的思想。

3. 项目维护

发布并运行程序，编写出软件使用说明书。

4. 写出专用周报告书

说明书要包含项目设计实现细节、专用周小结及参考文献。

5. 准备专用周检查。

四、专用周时间安排

序号	项目	学时数			备注
		讲授	设计或其他	合计	
1	布置准备、阅读资料	1	2	3	
2	网站设计策划	1	2	3	
3	网站设计	6	36	42	
4	编写专用周报告书	2	4	6	
5	项目检查		2	2	
8	合计	10	46	56	

开始日期：2025 年 4 月 21 日 **完成日期：2025 年 4 月 30 日**

指导老师：（已脱敏）（已脱敏）（已脱敏）

蜜雪冰城点单系统

一、系统分析

1、系统概述

蜜雪冰城点单系统，模拟了通过线上小程序点单，旨在为用户提供更便利的点单方式，用户可以不在店铺，只在手机上就可以实现菜单浏览和点单。该系统主要基于 vue3 框架开发。

系统主要功能包括：首页展示、会员管理、门店查询、点餐系统和订单管理等。用户可通过首页在线点餐，选择"到店自取"或"雪王外送"两种点餐方式。系统支持地址管理功能，用户可添加、编辑和删除配送地址。门店页面展示了附近门店信息，包括距离、营业状态等关键信息。

在 UI 设计方面，系统采用蜜雪冰城品牌特色的红白相间配色方案，各功能页面交互流畅，布局合理。技术上采用组件化开发思想，实现了页面间数据共享和状态管理，提高了代码复用率。数据存储采用本地存储实现用户信息、订单数据和地址信息的持久化。

该系统还实现了订单流程的全生命周期管理，包括下单、支付、配送和订单状态查询等环节。用户还可通过会员页面查看会员积分和相关权益。整体设计符合移动应用的交互规范，用户体验良好，界面简洁明了，操作流程清晰直观。

2、开发平台及工具选择

（1）Vue.js

Vue.js 是一款专为开发用户界面而设计的渐进式 JavaScript 框架，它提供了灵活的组件化开发模式，支持从简单页面到复杂应用的分层式构建。在 uni-app 开发中，Vue 的响应式数据绑定和模板语法让页面元素与数据紧密结合，通过组件化设计实现页面的模块化开发，只需关注单个文件中的模板、样式和逻辑，即可快速搭建出支持多端运行的跨平台应用，极大提升了开发效率和代码可维护性。

（2）VS code

VS Code 是微软开发的轻量级、跨平台代码编辑器，支持多种编程语言和丰富插件扩展。凭借智能代码提示、语法高亮、一键调试等功能，它为开发者提供了流畅的编码体验。在 uni-app 开发中，通过安装 Vue 、Uni-app 等插件，可实现代码自动补全、模板预览等便捷功能，提升开发效率。VS Code 是微软开发的轻量级、跨平台代码编辑器，支持多种编程语言和丰富插件扩展。凭借智能代码提示、语法高亮、一键调试等功能，它为开发者提供了流畅的编码体验。在 uni-app 开发中，通过安装 Vue 、 Uni-app 等插件，可实现代码自动补全、模板预览等便捷功能，提升开发效率。

（3）微信开发者工具

微信开发者工具是微信官方推出的一站式开发平台，集成了小程序、公众号、小游戏等多端开发功能。它内置模拟器可实时预览效果，支持微信登录、支付等开放接口，并提供调试工具与性能分析功能，方便开

发者高效构建微信生态应用。

（4）JavaScript

JavaScript 是一种直译式、弱类型、基于原型的脚本语言，主要用于网页交互开发。它支持多种编程范式，如面向对象、命令式等，可在浏览器和服务器端（如 Node.js 环境）运行。自 1995 年由 Brendan Eich 创造以来，经过 ES6 等版本更新，新增了箭头函数、模块导入等功能，使其功能更强大、表达更简洁，成为 web 开发的核心语言之一。

（5）CSS

CSS 作为层叠样式表语言，用于精确控制网页元素的外观和布局。它能定义字体、颜色、间距等样式属性，并通过盒模型、 flex 布局等实现复杂页面布局。CSS 还支持动态效果，如过渡、动画等，配合 JavaScript 可实现更丰富的交互体验，让网页从静态展示升级为动态、可视化的界面。

（6）HTML

HTML（超文本标记语言）是 web 页面的基础，由一系列标签组成，用于定义网页结构。标签如 、 等标记文本、图片、链接等元素，形成网页骨架。HTML5 的推出进一步扩展了其功能，新增了 、 等语义化标签及多媒体支持，使网页更具表现力和可访问性。

（7）Node.js

Node.js 是基于 Chrome V8 引擎的 JavaScript 运行时环境，采用事件驱动、非阻塞 I/O 模型。它使 JavaScript 能在服务器端运行，用于构建高性能网络应用，如 API 服务器、实时聊天应用等。凭借 npm 包管理器，开发者可便捷地使用和共享各种模块，丰富了开发生态。

二、功能分析

1、功能模块划分

（1）首页

本栏目能够自行选择到店自取还是雪王外送，还可以查看用户的登录信息，如果没有登录，会显示授权登录。除此之外，还会展示关于公司的活动和广告。

（2）点餐

本栏目有左侧导航栏供客户选择想要的奶茶分类，右侧的顶部又商品轮播图，右侧下方为商品详情。客户点击商品还能跳转到现商品详情页面。在商品详情页面可以选择温度、甜度、小料以及数量，选择后可以添加到购物车。在点餐页面点击去结算，可以支付成功并跳转到订单详情页面。

（3）订单

本页面可以查看历史订单记录，点击某个订单，可以进入详情页面。

（4）我的

本页面提供登录链接，实现查看雪王币、优惠券的功能。

（5）登录、注册

本页面可以实现注册和登录功能，并且有字段检查功能。

（6）收货地址

本栏目能够查看已有地址。还提供修改地址和新增地址的链接。

（7）店铺

可以查看当前是外卖还是自取，并且可以查看店铺的详情信息，并进行导航。

2、系统功能模块图

根据功能模块划分，绘制系统功能模块图，如下图 1 所示。

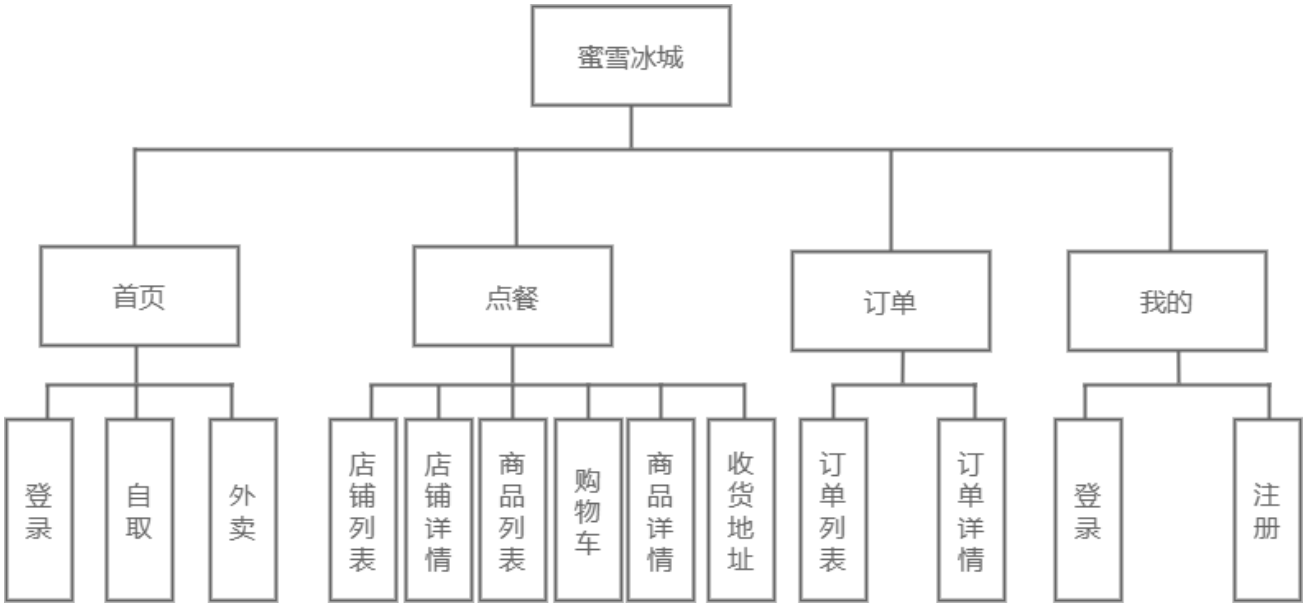


图 1 系统功能模块图

三、uni-app 程序项目架构图

本程序项目架构图，如下图 2 所示。

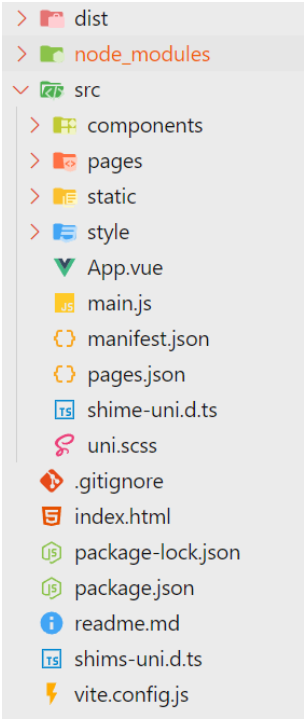


图 2 项目程序架构图

四、程序的制作风格介绍

蜜雪冰城点单系统主要的就是打通“浏览-点餐-支付-配送”等各个环节，实现线上线下一体化的数字运营体系。自主设计页面、创建精美的菜单界面和结构；要有覆盖面广、信息量大、可提供便利的全程人性化服务。从色彩、排版、特效三个方面来确定该网站的风格定位。

1. **色彩**

如果一个小程序的色彩鲜明，很容易引人注意，会给用户耳目一新的感觉。要有与众不同的色彩，界面的用色必须要有自己独特的风格，这样才能给用户留下深刻的印象。本项目选定红色为主色调，通过使用鲜亮的红色、纯净的白色、浅灰色及少量的黑色，使整个界面色彩统一，富有品牌特色。红色象征着活力与热情，与蜜雪冰城的品牌形象高度契合，而白色则增添了清爽感。其次，通过首页轮播图和商品展示图片以达到色彩的对比、平衡，避免产生单调感，同时也能突出产品的诱人质感。

2. **排版**

本小程序采用了标准的 TabBar 底部导航排版形式，通过底部导航将应用总体划分为首页、点餐、订单和我的四个主要功能区块。在各个页面中，采用了卡片式模块化布局，信息层次分明，重点突出。首页的会员区域、点餐入口和促销 banner 排版紧凑有序；点餐页面通过分类列表和商品网格实现了内容的有效组织；订单页面则按时间轴方式清晰展示订单状态。整体排版遵循移动端的操作习惯，考虑了用户单手操作的便捷性，重要功能按钮都位于易触达区域。

3. **特效**

动效可以清晰地表明各种功能模块之间的逻辑关系，即使在信息高度密集的情况下也能使界面从视觉上

保持良好的组织性。本项目在特效设计上秉持"简约而不简单"的原则，加入了适度的交互反馈：如轮播图的平滑过渡效果、页面切换的自然过渡动画等。这些特效既能提升用户体验，又不会造成视觉疲劳。特别是在商品加入购物车时的微妙动效，既满足了用户对操作结果的确认需求，又增添了趣味性。整体特效设计恰到好处，既显示了品牌活力，又保持了界面的专业性和高效性。

五、系统实施

1、首页

首先要做的是首页，我在顶部制作了一个轮播图，展示公司最新的活动，可以吸引用户的注意。在轮播图的下方，我添加了用户的登录信息，如果没有登录，会显示授权登录。在首页添加登录按钮，可以让用户的购买更加方便。在页面的中间，有两个按钮，分别为“到店自取”和“雪王外送”，点击后可以跳转到自取和外卖页面。效果如下图所示：



图 3 首页未登录效果



图 4 首页登陆后效果

实现的核心代码如下：

```
<script>
import ForceSwiper from "../../components/force-swiper/index.vue";
import ProgressBar from "../../components/progress-bar/index.vue";
```

```
export default {
  name: "IndexPage",
  components: { ForceSwiper, ProgressBar },
  data() {
    return {
      swiperList: [
        { imageUri: "/static/carousel/轮播图 1.gif", type: "url", target: "" },
        { imageUri: "/static/carousel/轮播图 2.jpg", type: "url", target: "" },
        { imageUri: "/static/carousel/轮播图 3.gif", type: "url", target: "" },
        { imageUri: "/static/carousel/轮播图 4.gif", type: "url", target: "" },
      ],
      current: 0,
      isLogin: false,
      user: null,
      i: 0,
    };
  },
  created() {
    // 页面创建时检查登录状态
    this.checkLoginStatus();
  },
  onShow() {
    // 每次显示页面时检查登录状态
    this.checkLoginStatus();
  },
  methods: {
    checkLoginStatus() {
      try {
        const storedUserInfo = uni.getStorageSync("userInfo");
        const storedIsLogin = uni.getStorageSync("isLogin");

        if (storedIsLogin === "true" && storedUserInfo) {
          this.isLogin = true;
          this.user = JSON.parse(storedUserInfo);
        } else {
          this.isLogin = false;
          this.user = null;
        }
      } catch (e) {
        console.error("读取登录状态出错:", e);
        this.isLogin = false;
        this.user = null;
      }
    },
    onSwiperChange(e) {
      this.current = e.detail.current;
    },
```

```
    gotoLogin() {
      // 跳转到登录注册页
      uni.navigateTo({
        url: "/pages/list/login/index",
      });
    },
    onMenuCardClick(type) {
      // 保存点餐类型（自取或外送）
      uni.setStorageSync("orderType", type);

      // 使用 switchTab 跳转到 tabBar 页面
      uni.switchTab({
        url: "/pages/list/store/index",
      });
    },
    onArticleClick() {
      uni.navigateTo({
        url: `/pages/web-view/index?url=http://mxbc.com/`,
      });
    },
    onSwiperTab(item) {
      if (item.type === "url") {
        uni.navigateTo({
          url: `/pages/web-view/index?url=${item.target}`,
        });
      } else {
        uni.navigateTo({
          url: `/pages/product/detail?id=${item.target}`,
        });
      }
    },
  },
};
</script>
```

2．店铺

其次要做的是店铺页面。不论是通过首页进入点餐，还是点击底部的导航栏进入点餐，都会先进入店铺选择页面。因为只有选择了店铺后，才可以点单。在首页，如果点击“到店自取”，会进入到自取店铺页面。如果点击“雪王外送”，会进入外卖店铺页面，外卖店铺页面顶部会有外送地址。实现方法为点击时，会将状态保存在本地存储中，店铺页面根据本地存储的变量来判断是否渲染地址部分。点击店铺，可以展示店铺详情信息弹窗，点击弹窗的导航，可以跳转到地图页面，此方法调用了 uni-app 的地图导航 API。效果如下图所示：

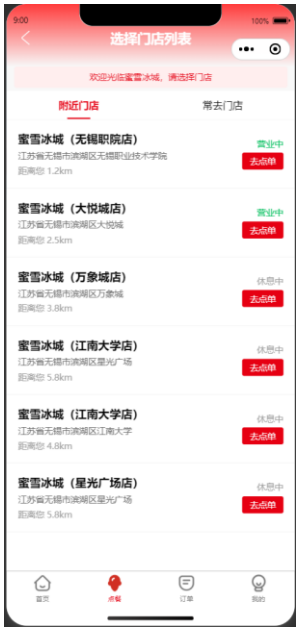


图5 自取店铺页面



图6 外卖店铺页面

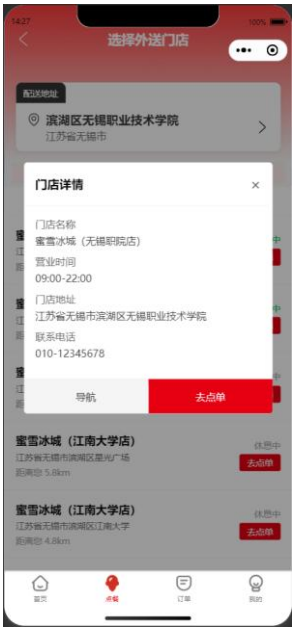


图7 店铺详情信息



图8 店铺导航页面

实现的核心代码如下：

```
<script>
export default {
  name: "StorePage",
  data() {
    return {
      currentTab: 0,
      searchValue: "",
      isLoading: true,
      orderType: "到店自取",
      isDeliveryMode: false,
      tabs: [{ title: "附近门店" }, { title: "常去门店" }],
      selectedAddress: { id: "1", name: "谢凡", phone: "11111111111", address: "江苏省无锡市滨湖区无锡职业技术学院" },
      storeList: [
        { id: "1", name: "蜜雪冰城（无锡职院店）", address: "江苏省无锡市滨湖区无锡职业技术学院", distance: 1.2, isOpen: true, businessHours: "09:00-22:00", phone: "010-12345678", latitude: 39.9, longitude: 116.3 },
        { id: "2", name: "蜜雪冰城（大悦城店）", address: "江苏省无锡市滨湖区大悦城", distance: 2.5, isOpen: true, businessHours: "09:00-21:30", phone: "010-87654321", latitude: 39.95, longitude: 116.35 },
        { id: "3", name: "蜜雪冰城（万象城店）", address: "江苏省无锡市滨湖区万象城", distance: 3.8, isOpen: false, businessHours: "10:00-22:00", phone: "010-11223344", latitude: 39.92, longitude: 116.32 },
        { id: "4", name: "蜜雪冰城（江南大学店）", address: "江苏省无锡市滨湖区星光广场", distance: 5.8, isOpen: false, businessHours: "10:00-22:00", phone: "010-16523344", latitude: 39.92, longitude: 116.32 },
        { id: "5", name: "蜜雪冰城（江南大学店）", address: "江苏省无锡市滨湖区江南大学", distance: 4.8, isOpen: false, businessHours: "10:00-22:00", phone: "010-16523344", latitude: 39.92, longitude: 116.32 },
        { id: "5", name: "蜜雪冰城（星光广场店）", address: "江苏省无锡市滨湖区星光广场", distance: 5.8, isOpen: false, businessHours: "10:00-22:00", phone: "010-16523344", latitude: 39.92, longitude: 116.32 }
      ],
      currentStore: { id: "", name: "", address: "", distance: 0, isOpen: true, businessHours: "", phone: "", latitude: 0,
```

```
longitude: 0 }
      };
  },
  computed: {
    hasAddress() {
      return this.selectedAddress && this.selectedAddress.address;
    },
    currentAddressDetail() {
      if (!this.hasAddress) return "";
      const address = this.selectedAddress.address;
      const parts = address.split("市");
      return parts.length > 1 ? parts[1] : address;
    },
    currentAddressCity() {
      if (!this.hasAddress) return "";
      const address = this.selectedAddress.address;
      const parts = address.split("市");
      return parts.length > 1 ? `${parts[0]}市` : "";
    },
  },
  onLoad(options) {
    if (this.storeList.length > 0) {
      this.currentStore = this.storeList[0];
      uni.setStorageSync("selectedStore", JSON.stringify(this.currentStore));
    }
    this.getStoreList();
    this.initAddressList();
    this.checkOrderType();
    uni.$on("addressChanged", (address) => {
      if (address) this.selectedAddress = JSON.parse(JSON.stringify(address));
    });
  },
  onShow() {
    this.checkOrderType();
    this.refreshAddressInfo();
  },
  onUnload() {
    uni.$off("addressChanged");
  },
  methods: {
    checkOrderType() {
      const orderType = uni.getStorageSync("orderType") || "";
      this.isDeliveryMode = orderType.indexOf("外送") !== -1;
      if (this.isDeliveryMode) this.initAddressList();
    },
    initAddressList() {
      if (this.isDeliveryMode) {
```

```
        try {
          const addressListStr = uni.getStorageSync("addressList");
          let addressList = addressListStr ? JSON.parse(addressListStr) : [];
          let selectedAddressId = uni.getStorageSync("selectedAddressId");

          if (selectedAddressId && addressList.length > 0) {
            const address = addressList.find(item => item.id === selectedAddressId);
            if (address) {
              this.selectedAddress = address;
            } else if (addressList.length > 0) {
              this.selectedAddress = addressList[0];
              uni.setStorageSync("selectedAddressId", addressList[0].id);
            } else {
              this.selectedAddress = {};
            }
          } else if (addressList.length > 0) {
            this.selectedAddress = addressList[0];
            uni.setStorageSync("selectedAddressId", addressList[0].id);
          } else {
            this.selectedAddress = {};
          }
        } catch (error) {
          this.selectedAddress = {};
        }
      }
    },
    refreshAddressInfo() {
      try {
        const selectedAddressId = uni.getStorageSync("selectedAddressId");
        const addressListStr = uni.getStorageSync("addressList");

        if (!addressListStr) {
          this.selectedAddress = {};
          return;
        }

        const addressList = JSON.parse(addressListStr);
        if (addressList.length === 0) {
          this.selectedAddress = {};
          return;
        }

        if (selectedAddressId) {
          const address = addressList.find(item => item.id === selectedAddressId);
          if (address) {
            this.selectedAddress = JSON.parse(JSON.stringify(address));
            return;
```

```
            }
        }

        if (addressList.length > 0) {
            this.selectedAddress = JSON.parse(JSON.stringify(addressList[0]));
            uni.setStorageSync("selectedAddressId", addressList[0].id);
        } else {
            this.selectedAddress = {};
        }
    } catch (error) {
        this.selectedAddress = {};
    }
},
getStoreList() {},
switchTab(index) {
    this.currentTab = index;
},
popupStoreDetail(store) {
    this.currentStore = store;
    this.$refs.storeDetailPopup.open();
},
closeStoreDetail() {
    this.$refs.storeDetailPopup.close();
},
gotoMenu(store) {
    const storeToUse = store || this.currentStore;
    uni.setStorageSync("selectedStore", JSON.stringify(storeToUse));
    uni.switchTab({
        url: "/pages/list/menu/index",
        fail: () => {
            uni.navigateTo({
                url: `/pages/list/menu/index`,
            });
        },
    });
},
navigateToStore() {
    uni.openLocation({
        latitude: this.currentStore.latitude,
        longitude: this.currentStore.longitude,
        name: this.currentStore.name,
        address: this.currentStore.address,
    });
},
goToAddressList() {
    uni.navigateTo({
        url: "/pages/list/address/index",
```

```
            events: {
                addressSelected: function (data) {
                    console.log("收到地址选择事件:", data);
                },
            },
            success: () => {
                console.log("跳转到地址列表页面");
            },
        });
    },
    goBack() {
        uni.navigateBack({
            fail: () => {
                uni.switchTab({
                    url: "/pages/index/index",
                });
            },
        });
    },
  },
};
</script>
```

3．收货地址

收获地址页面可以进行对地址的编辑、删除，以及添加新地址。添加的新地址会将数据保存在本地浏览器，编辑地址会对本地的数据进行修改，此操作防止刷新数据丢失，提升了用户的体验性。效果如下图所示：

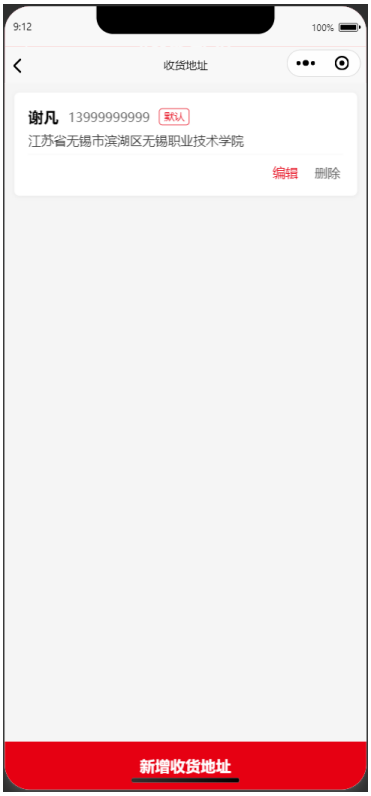


图 9 地址管理页面



图 10 地址编辑与添加页面



图 11 添加地址后的店铺首页

实现的核心代码如下：

```
<script>
export default {
  data() {
    return {
      addressList: []
    }
  },
  onLoad() {
    // 加载地址列表数据
    this.loadAddressList();
  },
  onShow() {
    // 每次显示页面时重新加载数据
    this.loadAddressList();
  },
  methods: {
```

```
// 加载地址列表
loadAddressList() {
  try {
    const addressListStr = uni.getStorageSync('addressList');
    if (addressListStr) {
      this.addressList = JSON.parse(addressListStr);
    }
  } catch (error) {
    console.error('加载地址列表失败:', error);
    uni.showToast({
      title: '加载地址失败',
      icon: 'none'
    });
  }
},

// 选择地址
selectAddress(id) {
  try {
    // 获取选中的地址对象
    const selectedAddress = this.addressList.find(item => item.id === id);

    if (selectedAddress) {
      console.log('选择地址:', selectedAddress);

      // 存储选中的地址 ID
      uni.setStorageSync('selectedAddressId', id);

      // 获取页面实例，准备返回
      const pages = getCurrentPages();
      if (pages.length > 1) {
```

```
            // 获取上一页实例
            const prevPage = pages[pages.length - 2];

            // 如果上一页有 refreshAddressInfo 方法，直接调用
            if (prevPage.$vm && typeof prevPage.$vm.refreshAddressInfo === 'function') {
              // 尝试直接修改上一页的数据
              prevPage.$vm.selectedAddress = JSON.parse(JSON.stringify(selectedAddress));
              console.log('已直接设置上一页的地址:', selectedAddress);
            }

            // 返回上一页
            uni.navigateBack({
              success: () => {
                console.log('返回上一页成功');
                // 额外发出一个事件通知
                uni.$emit('addressChanged', selectedAddress);
              }
            });
          }
        } else {
          uni.showToast({
            title: '地址数据有误',
            icon: 'none'
          });
        }
      } catch (error) {
        console.error('选择地址失败:', error);
        uni.showToast({
          title: '选择地址失败',
          icon: 'none'
        });
```

```
    }
  },

  // 跳转到地址详情页
  gotoAddressDetail(id) {
    uni.navigateTo({
      url: `/pages/list/address/detail?id=${id}`
    });
  },

  // 跳转到新增地址页面
  addNewAddress() {
    uni.navigateTo({
      url: '/pages/list/address/edit'
    });
  },

  // 编辑地址
  editAddress(id) {
    uni.navigateTo({
      url: `/pages/list/address/edit?id=${id}`
    });
  },

  // 删除地址
  deleteAddress(id) {
    uni.showModal({
      title: '提示',
      content: '确定要删除此地址吗？',
      success: (res) => {
        if (res.confirm) {
```

```
// 从地址列表中移除该地址
const newAddressList = this.addressList.filter(item => item.id !== id);

// 更新本地存储
uni.setStorageSync('addressList', JSON.stringify(newAddressList));

// 如果删除的是当前选中的地址，则更新选中的地址
const selectedAddressId = uni.getStorageSync('selectedAddressId');
if (selectedAddressId === id) {
  // 如果还有其他地址，选择第一个作为默认地址
  if (newAddressList.length > 0) {
    // 优先选择默认地址
    const defaultAddress = newAddressList.find(item => item.isDefault);
    if (defaultAddress) {
      uni.setStorageSync('selectedAddressId', defaultAddress.id);
    } else {
      uni.setStorageSync('selectedAddressId', newAddressList[0].id);
    }
  } else {
    // 如果没有地址了，清除选中的地址 ID
    uni.removeStorageSync('selectedAddressId');
  }
}

// 更新页面数据
this.addressList = newAddressList;

// 提示删除成功
uni.showToast({
  title: '删除成功',
  icon: 'success'
```

```
                });
            }
        }
    });
  }
}
</script>
```

4. 点餐

（1）分类

接着要做的是点餐分类。在分类页面，左侧可以点击不同的种类进行切换，右侧顶部有搜索功能，可以实现模糊查询。搜索下方为店铺信息，点击会跳转到店铺选择页面，可以重新选择店铺。店铺区域下方为广告轮播图。剩下区域为商品详情。点击底部的结算可以成功支付并跳转到订单详情页面。效果如下图所示：



图 12 分类页面



图 13 模糊搜索效果

实现的核心代码如下：

```
<script>
import Sidebar from "../../../components/sidebar/index.vue";
import CartBar from "../../../components/cart-bar/index.vue";
```

```
export default {
  components: {
    Sidebar,
    CartBar,
  },
  data() {
    return {
      headerStyle: "",
      searchKeyword: "", // 搜索关键词
      cartItems: [], // 购物车商品
      totalPrice: 0, // 购物车总价
      swiperList: [
        {
          imageUri: "../../../static/carousel/IMG_1508.JPG",
          type: "url",
          target: "",
        },
        {
          imageUri: "../../../static/carousel/IMG_1508.JPG",
          type: "product",
          target: "1",
        },
        {
          imageUri: "../../../static/carousel/IMG_1508.JPG",
          type: "url",
          target: "",
        },
      ],
      current: 0,
      currentCategoryIndex: 0,
      categoryList: ["鲜萃咖啡","当季新品","本店热销","真鲜果茶","醇香奶茶","轻乳系列"],
      goodsList: [
        // 鲜萃咖啡
        {
          id: 1,
          categoryId: 0,
          name: "生椰拿铁",
          description: "醇香咖啡搭配香浓生椰乳",
          price: 13,
          image: "/static/images/product-image.png",
        },
        {
          id: 2,
          categoryId: 0,
          name: "冰萃咖啡",
          description: "精选阿拉比卡咖啡豆现萃",
```

```
    price: 11,
    image: "/static/images/product-image.png",
  },
  {
    id: 3,
    categoryId: 0,
    name: "美式咖啡",
    description: "醇香浓郁，回味甘甜",
    price: 10,
    image: "/static/images/product-image.png",
  },
  // 当季新品
  {
    id: 4,
    categoryId: 1,
    name: "芝芝莓莓",
    description: "莓香浓郁，口感丰富",
    price: 15,
    image: "/static/images/product-image.png",
  },
  {
    id: 5,
    categoryId: 1,
    name: "多肉葡萄",
    description: "多重果肉，鲜爽可口",
    price: 14,
    image: "/static/images/product-image.png",
  },
  // 本店热销
  {
    id: 6,
    categoryId: 2,
    name: "蜜雪冰城",
    description: "招牌饮品，清爽怡人",
    price: 6,
    image: "/static/images/product-image.png",
  },
  {
    id: 7,
    categoryId: 2,
    name: "柠檬水",
    description: "新鲜柠檬，酸甜可口",
    price: 5,
    image: "/static/images/product-image.png",
  },
  // 真鲜果茶
  {
```

```
    id: 8,
    categoryId: 3,
    name: "满杯百香果",
    description: "酸甜百香果，清爽解腻",
    price: 12,
    image: "/static/images/product-image.png",
  },
  {
    id: 9,
    categoryId: 3,
    name: "杨枝甘露",
    description: "经典港式饮品，香甜软糯",
    price: 14,
    image: "/static/images/product-image.png",
  },
  // 醇香奶茶
  {
    id: 10,
    categoryId: 4,
    name: "蜜香奶茶",
    description: "醇香奶茶，香浓丝滑",
    price: 9,
    image: "/static/images/product-image.png",
  },
  {
    id: 11,
    categoryId: 4,
    name: "椰香奶茶",
    description: "椰香浓郁，奶味十足",
    price: 11,
    image: "/static/images/product-image.png",
  },
  // 轻乳系列
  {
    id: 12,
    categoryId: 5,
    name: "芝士轻乳",
    description: "轻盈奶香，芝士醇厚",
    price: 13,
    image: "/static/images/product-image.png",
  },
  {
    id: 13,
    categoryId: 5,
    name: "芒果轻乳",
    description: "芒果清甜，奶香浓郁",
    price: 14,
```

```
            image: "/static/images/product-image.png",
          },
        ],
        currentStore: {
          id: "",
          name: "请选择门店",
          address: "",
          distance: 0,
          isOpen: true,
          businessHours: "",
          phone: "",
          latitude: 0,
          longitude: 0,
        },
      };
    },
    computed: {
      // 获取当前分类名称
      currentCategory() {
        return this.categoryList[this.currentCategoryIndex];
      },
      // 根据当前分类和搜索关键词筛选商品列表
      filteredGoodsList() {
        let result = this.goodsList.filter(
          (item) => item.categoryId === this.currentCategoryIndex
        );

        // 如果有搜索关键词，进行模糊匹配
        if (this.searchKeyword.trim()) {
          const keyword = this.searchKeyword.toLowerCase().trim();
          result = result.filter(
            (item) =>
              item.name.toLowerCase().includes(keyword) ||
              item.description.toLowerCase().includes(keyword)
          );
        }

        return result;
      },
    },
    onLoad(options) {
      this.makeHeaderStyle();
      // 尝试从 URL 参数获取门店信息
      if (options.storeInfo) {
        try {
          const storeInfo = JSON.parse(decodeURIComponent(options.storeInfo));
          this.currentStore = storeInfo;
```

```
        } catch (error) {
          this.getStoreFromStorage();
        }
      } else {
        this.getStoreFromStorage();
      }
    },
    onShow() {
      this.getStoreFromStorage();
      this.checkReorderItems();
      this.checkTempCartItems();
    },
    methods: {
      checkReorderItems() {
        try {
          const reorderItemsStr = uni.getStorageSync("reorderItems");
          if (reorderItemsStr) {
            const reorderItems = JSON.parse(reorderItemsStr);
            if (reorderItems && reorderItems.length > 0) {
              this.cartItems = [];
              reorderItems.forEach((item) => {
                this.addToCart(item, false);
              });
              uni.removeStorageSync("reorderItems");
            }
          }
        } catch (error) {}
      },
      getStoreFromStorage() {
        try {
          const storeInfo = uni.getStorageSync("selectedStore");
          if (storeInfo) {
            this.currentStore = JSON.parse(storeInfo);
          }
        } catch (error) {}
      },
      makeHeaderStyle() {
        const menuButtonInfo = uni.getMenuButtonBoundingClientRect();
        const { top, height } = menuButtonInfo;
        const menuButtonCenterPoint = top + height / 2;
        this.headerStyle = `margin-top: calc(${menuButtonCenterPoint}px - 32rpx);`;
      },
      switchCurrentStore() {
        uni.switchTab({
          url: "/pages/list/store/index",
        });
      },
```

```
onSwiperChange(e) {
  this.current = e.detail.current;
},
onSwiperTab(item) {
  if (item.type === "url") {
    uni.navigateTo({
      url: `/pages/web-view/index?url=${item.target}`,
    });
  } else {
    uni.navigateTo({
      url: `/pages/product/detail?id=${item.target}`,
    });
  }
},
onCategoryChange(index) {
  this.currentCategoryIndex = index;
},
onSearchInput() {},
addToCart(item, showToast = true) {
  const existingItem = this.cartItems.find(
    (cartItem) => cartItem.id === item.id && !cartItem.customized
  );

  if (existingItem) {
    existingItem.quantity += 1;
  } else {
    this.cartItems.push({
      ...item,
      quantity: 1,
      customized: false,
    });
  }

  this.updateTotalPrice();

  if (showToast) {
    uni.showToast({
      title: "已添加到购物车",
      icon: "success",
      duration: 1500,
    });
  }
},
addCustomizedItemToCart(item) {
  const customizedItem = {
    ...item,
    customized: true,
```

```
        uniqueId: Date.now(),
      };
      this.cartItems.push(customizedItem);
      this.updateTotalPrice();
    },
    checkTempCartItems() {
      try {
        const tempCartItemsStr = uni.getStorageSync("tempCartItems");
        if (tempCartItemsStr) {
          const tempCartItems = JSON.parse(tempCartItemsStr);
          if (tempCartItems && tempCartItems.length > 0) {
            tempCartItems.forEach((item) => {
              this.addCustomizedItemToCart(item);
            });
            uni.removeStorageSync("tempCartItems");
          }
        }
      } catch (error) {}
    },
    goToProductDetail(item) {
      try {
        uni.setStorageSync("allProducts", JSON.stringify(this.goodsList));
        uni.setStorageSync("currentProduct", JSON.stringify(item));
        uni.navigateTo({
          url: `/pages/product/detail?id=${item.id}`,
        });
      } catch (error) {
        uni.navigateTo({
          url: `/pages/product/detail?id=${item.id}`,
        });
      }
    },
    updateTotalPrice() {
      this.totalPrice = this.cartItems.reduce((total, item) => {
        if (item.customized && item.totalPrice) {
          return total + item.totalPrice;
        } else {
          return total + item.price * item.quantity;
        }
      }, 0);
    },
    onCheckout() {
      if (this.cartItems.length === 0) {
        uni.showToast({
          title: "购物车为空",
          icon: "none",
          duration: 1500,
```

```
    });
    return;
}

const userInfoStr = uni.getStorageSync("userInfo");
let userName = "匿名用户";
let userPhone = "未绑定手机号";

if (userInfoStr) {
    try {
        const userInfo = JSON.parse(userInfoStr);
        if (userInfo.desensitiveMobile) {
            userName = userInfo.desensitiveMobile;
        }
        if (userInfo.mobile) {
            userPhone = userInfo.mobile;
        }
    } catch (error) {}
}

const orderInfo = {
    id: Date.now().toString(),
    items: [...this.cartItems],
    totalPrice: this.totalPrice,
    status: "进行中",
    time: new Date().toLocaleString(),
    store: this.currentStore.name || "未知门店",
    address: this.currentStore.address || "未知地址",
    userName: userName,
    userPhone: userPhone,
};

let orderList = uni.getStorageSync("orderList") || [];
orderList.unshift(orderInfo);
uni.setStorageSync("orderList", orderList);

this.cartItems = [];
this.totalPrice = 0;

uni.showToast({
    title: "支付成功",
    icon: "success",
    duration: 1000,
});

setTimeout(() => {
    uni.navigateTo({
```

```
            url: `/pages/list/order/detail?id=${orderInfo.id}`,
          });
        }, 1200);
      },
    },
};
</script>
```

（2）商品详情页面

然后我们做的是点餐的商品详情页面，点击单个商品，会跳转到商品详情页面，可以选择商品的甜度、温度以及添加小料。运行效果图，如下所示：



图 14 商品详情页面

实现的核心代码如下：

```
<script>
export default {
  data() {
    return {
      productId: '',
      product: { id: '', name: '', description: '', price: 0, image: '/static/images/product-image.png' },
      quantity: 1,
      temperatureOptions: ['热饮', '常温', '去冰', '少冰', '多冰'],
      selectedTemperature: '常温',
      sweetnessOptions: ['无糖', '微糖', '半糖', '少糖', '标准糖'],
```

```
      selectedSweetness: '标准糖',
      toppingsOptions: [
        { name: '珍珠', price: 2 },
        { name: '椰果', price: 2 },
        { name: '芋圆', price: 3 },
        { name: '红豆', price: 2 },
        { name: '奶盖', price: 5 }
      ],
      selectedToppings: []
    };
  },
  computed: {
    totalPrice() {
      let toppingsPrice = 0;
      this.selectedToppings.forEach(topping => {
        const toppingItem = this.toppingsOptions.find(item => item.name === topping);
        if (toppingItem) toppingsPrice += toppingItem.price;
      });
      return (this.product.price + toppingsPrice) * this.quantity;
    }
  },
  onLoad(options) {
    if (options && options.id) {
      this.productId = options.id;
      this.fetchProductDetail();
    } else {
      uni.showToast({ title: '商品信息获取失败', icon: 'none', duration: 2000 });
      setTimeout(() => uni.navigateBack(), 2000);
    }
  },
  methods: {
    fetchProductDetail() {
      uni.showLoading({ title: '加载中...' });
      try {
        const currentProductStr = uni.getStorageSync('currentProduct');
        if (currentProductStr) {
          const currentProduct = JSON.parse(currentProductStr);
          if (currentProduct.id.toString() === this.productId.toString()) {
            if (currentProduct.image && !currentProduct.image.startsWith('/')) {
              currentProduct.image = '/' + currentProduct.image.replace(/^\.\.\/\.\.\//, '');
            }
            this.product = currentProduct;
            uni.hideLoading();
            return;
          }
        }
      } catch (error) {}
```

```
        try {
          const allProductsStr = uni.getStorageSync('allProducts');
          if (allProductsStr) {
            const productList = JSON.parse(allProductsStr);
            const product = productList.find(item => item.id.toString() === this.productId.toString());
            if (product) {
              if (product.image && !product.image.startsWith('/')) {
                product.image = '/' + product.image.replace(/^\.\.\/\.\.\//, '');
              }
              this.product = product;
              uni.hideLoading();
              return;
            }
          }
        } catch (error) {}

        const mockProducts = [
          { id: 1, name: "生椰拿铁", description: "醇香咖啡搭配香浓生椰乳", price: 13, image: "/static/images/product-image.png" },
          { id: 2, name: "冰萃咖啡", description: "精选阿拉比卡咖啡豆现萃", price: 11, image: "/static/images/product-image.png" },
          { id: 3, name: "美式咖啡", description: "醇香浓郁，回味甘甜", price: 10, image: "/static/images/product-image.png" },
          { id: 4, name: "芝芝莓莓", description: "莓香浓郁，口感丰富", price: 15, image: "/static/images/product-image.png" },
          { id: 5, name: "多肉葡萄", description: "多重果肉，鲜爽可口", price: 14, image: "/static/images/product-image.png" },
          { id: 6, name: "蜜雪冰城", description: "招牌饮品，清爽怡人", price: 6, image: "/static/images/product-image.png" }
        ];

        const mockProduct = mockProducts.find(item => item.id.toString() === this.productId.toString());
        if (mockProduct) {
          this.product = mockProduct;
          uni.hideLoading();
        }
      },
      increaseQuantity() { this.quantity++; },
      decreaseQuantity() { if (this.quantity > 1) this.quantity--; },
      toggleTopping(item) {
        const index = this.selectedToppings.indexOf(item.name);
        if (index > -1) this.selectedToppings.splice(index, 1);
        else this.selectedToppings.push(item.name);
      },
      addToCart() {
        let toppingsPrice = 0;
```

```
        const selectedToppingsList = [];
        this.selectedToppings.forEach(toppingName => {
          const topping = this.toppingsOptions.find(item => item.name === toppingName);
          if (topping) {
            toppingsPrice += topping.price;
            selectedToppingsList.push({ name: topping.name, price: topping.price });
          }
        });

        const cartItem = {
          id: this.product.id,
          name: this.product.name,
          description: this.product.description,
          price: this.product.price,
          image: this.product.image,
          quantity: this.quantity,
          temperature: this.selectedTemperature,
          sweetness: this.selectedSweetness,
          toppings: selectedToppingsList,
          toppingsPrice: toppingsPrice,
          totalPrice: this.totalPrice
        };

        const pages = getCurrentPages();
        const prevPage = pages[pages.length - 2];

        if (prevPage && prevPage.$vm.addCustomizedItemToCart) {
          prevPage.$vm.addCustomizedItemToCart(cartItem);
          uni.showToast({ title: '已添加到购物车', icon: 'success' });
          setTimeout(() => uni.navigateBack(), 1500);
        } else {
          const tempCartItems = uni.getStorageSync('tempCartItems') || [];
          tempCartItems.push(cartItem);
          uni.setStorageSync('tempCartItems', tempCartItems);
          uni.showToast({ title: '已添加到购物车', icon: 'success' });
          setTimeout(() => uni.navigateBack(), 1500);
        }
      },
      handleImageError() {
        this.product.image = '/static/images/product-image.png';
      }
    }
};
</script>
```

（3）购物车

紧接着就是购物车页面，点击商品右侧的加号按钮或者商品详情页的“加入购物车”按钮，可以将商品

添加到购物车中。点击购物车中的减价可以实现对商品数量的增减，还可以实现清空购物车功能。运行效果图，如下所示：

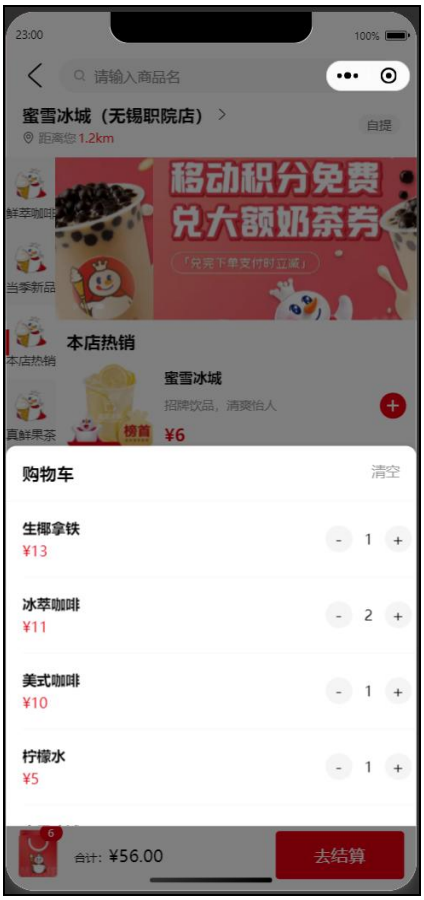


图 15 购物车界面

实现的核心代码如下：

```
<script>
export default {
  name: "CartBar",
  props: {
    cartItems: {
      type: Array,
      default: () => [],
    },
    totalPrice: {
      type: Number,
      default: 0,
    },
  },
  emits: ["update:cartItems", "update:totalPrice", "checkout"],
  data() {
    return {
      showCartPanel: false, // 控制购物车面板显示
    };
```

```
  },
  computed: {
    // 计算购物车中的商品总数
    totalItems() {
      return this.cartItems.reduce((total, item) => total + item.quantity, 0);
    },
  },
  methods: {
    // 结算按钮点击事件
    onCheckClick() {
      if (this.totalPrice <= 0) {
        uni.showToast({
          title: "购物车为空",
          icon: "none",
          duration: 1500,
        });
        return;
      }

      // 触发结算事件
      this.$emit("checkout");
    },

    // 切换购物车面板显示/隐藏
    toggleCartPanel() {
      this.showCartPanel = !this.showCartPanel;
    },

    // 清空购物车
    clearCart() {
      uni.showModal({
        title: "提示",
        content: "确定要清空购物车吗？",
        success: (res) => {
          if (res.confirm) {
            console.log("清空购物车，发送事件：update:cartItems", []);
            this.$emit("update:cartItems", []);
            this.$emit("update:totalPrice", 0);
            this.showCartPanel = false;
          }
        },
      });
    },

    // 增加商品数量
    increaseQuantity(item) {
      const updatedItems = [...this.cartItems];
```

```
    const index = updatedItems.findIndex((i) =>
      item.customized ? i.uniqueId === item.uniqueId : i.id === item.id && !i.customized
    );

    if (index !== -1) {
      updatedItems[index].quantity += 1;

      // 如果是定制商品，更新总价
      if (updatedItems[index].customized) {
        const unitPrice = updatedItems[index].totalPrice / updatedItems[index].quantity;
        updatedItems[index].totalPrice = unitPrice * (updatedItems[index].quantity + 1);
      }

      this.$emit("update:cartItems", updatedItems);
      this.updateTotalPrice(updatedItems);
    }
  },

  // 减少商品数量
  decreaseQuantity(item) {
    const updatedItems = [...this.cartItems];
    const index = updatedItems.findIndex((i) =>
      item.customized ? i.uniqueId === item.uniqueId : i.id === item.id && !i.customized
    );

    if (index !== -1) {
      if (updatedItems[index].quantity > 1) {
        updatedItems[index].quantity -= 1;

        if (updatedItems[index].customized) {
          const unitPrice = updatedItems[index].totalPrice / updatedItems[index].quantity;
          updatedItems[index].totalPrice = unitPrice * (updatedItems[index].quantity - 1);
        }
      } else {
        updatedItems.splice(index, 1);
      }

      this.$emit("update:cartItems", updatedItems);
      this.updateTotalPrice(updatedItems);
    }
  },

  // 更新总价
  updateTotalPrice(items) {
    const newTotal = items.reduce((total, item) => {
      if (item.customized) {
        return total + item.totalPrice;
```

```
            } else {
                return total + item.price * item.quantity;
            }
        }, 0);

        this.$emit("update:totalPrice", newTotal);
    },
  },
};
</script>
```

5. 订单

（1）订单列表

页面显示所有订单历史记录，点击单个记录，可以进入订单详情页面。运行效果图，如下所示：

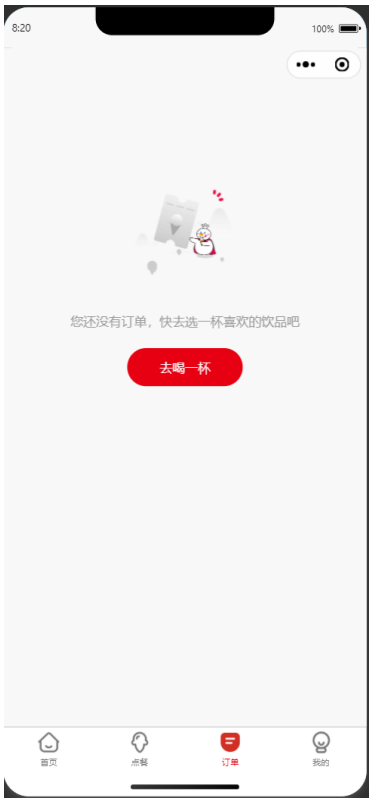


图 16 空订单页面



图 17 历史订单页面

实现的核心代码如下：

```
<script>
export default {
  name: "OrderPage",
  data() {
    return {
      current: 0,
      tabList: [{ name: "进行中" }, { name: "历史" }, { name: "退单" }],
      orderList: [],
    };
```

```
  },
  computed: {
    // 根据当前标签页筛选订单列表
    filteredOrderList() {
      if (this.current === 0) {
        // 进行中的订单
        return this.orderList.filter(order => order.status === "进行中");
      } else if (this.current === 1) {
        // 历史订单
        return this.orderList.filter(order => order.status === "已完成");
      } else {
        // 退单
        return this.orderList.filter(order => order.status === "已退单");
      }
    },
  },
  onShow() {
    // 每次显示页面时获取最新订单数据
    this.getOrderList();
  },
  methods: {
    onTabChange(index) {
      this.current = index;
    },
    goMenu() {
      // 跳转到菜单页面
      uni.switchTab({
        url: "/pages/list/menu/index",
      });
    },
    // 获取订单列表
    getOrderList() {
      // 从本地存储获取订单数据
      const orderList = uni.getStorageSync("orderList") || [];
      this.orderList = orderList;
    },
    goToOrderDetail(order) {
      // 跳转到订单详情页面
      uni.navigateTo({
        url: `/pages/list/order/detail?id=${order.id}`
      });
    },
  },
};
</script>
```

（2）订单详情页面

点击订单列表的单个订单，进入订单详情页面。运行效果图，如下所示：



图 18 订单详情页

实现的核心代码如下：

```
<script>
export default {
  data() {
    return {
      orderId: '',
      orderInfo: {
        id: '',
        status: '进行中',
        time: '',
        store: '',
        address: '',
        items: [],
        totalPrice: 0,
        userName: '匿名用户',
        userPhone: '未绑定手机号'
      }
    }
  },
  onLoad(options) {
    if (options.id) {
      this.orderId = options.id;
```

```
        this.getOrderDetail();
    } else if (options.orderInfo) {
        // 如果是直接传递了订单信息
        try {
            this.orderInfo = JSON.parse(decodeURIComponent(options.orderInfo));
        } catch (error) {
            console.error('解析订单信息失败', error);
            uni.showToast({
                title: '订单信息解析失败',
                icon: 'none'
            });
        }
    } else {
        uni.showToast({
            title: '未找到订单信息',
            icon: 'none'
        });
        setTimeout(() => {
            this.goBack();
        }, 1500);
    }
},
methods: {
    // 获取订单详情
    getOrderDetail() {
        const orderList = uni.getStorageSync('orderList') || [];
        const order = orderList.find(item => item.id === this.orderId);

        if (order) {
            this.orderInfo = order;
        } else {
            uni.showToast({
                title: '未找到该订单',
                icon: 'none'
            });
            setTimeout(() => {
                this.goBack();
            }, 1500);
        }
    },

    // 获取订单状态描述
    getStatusDescription() {
        const status = this.orderInfo.status;
        if (status === '进行中') {
            return '您的订单正在制作中，请耐心等待';
        } else if (status === '已完成') {
```

```
            return '您的订单已完成，感谢您的惠顾';
        } else if (status === '已退单') {
            return '该订单已退款，资金将原路返回';
        }
        return '';
    },

    // 返回订单列表
    goBack() {
        uni.navigateBack();
    },

    // 再来一单
    reorder() {
        // 将当前订单中的商品添加到购物车
        const cartItems = this.orderInfo.items.map(item => ({
            ...item,
            quantity: 1 // 默认数量设为 1
        }));

        // 保存到本地存储
        uni.setStorageSync('reorderItems', JSON.stringify(cartItems));

        // 跳转到菜单页
        uni.switchTab({
            url: '/pages/list/menu/index'
        });

        // 提示用户
        uni.showToast({
            title: '已将商品添加到购物车',
            icon: 'none'
        });
    }
  }
}
</script>
```

6．我的

我的个人中心界面，主要展示用户个人信息，点击顶部的授权登录会跳转到登录与注册页面，点击底部的退出账号可以退出登录。除此之外还可以查看自己的雪王币和优惠券数量。效果如下图所示：



图 19 我的页面未登录状态

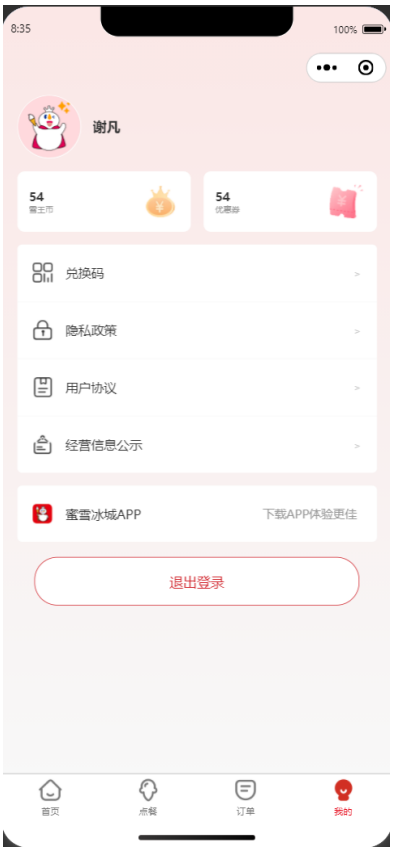


图 20 我的页面登录状态

实现的核心代码如下：

```
<script>
export default {
  name: "MinePage",
  data() {
    return {
      isLogin: false,
      user: null,
      userInfo: {
        nickName: "用户昵称",
        avatarUrl: "../../assets/images/me-avatar.png", // 默认头像或登录后更新
      },
      customPageUrls: { privacy: "", agreement: "", xuke: "" },
    };
  },
  created() {
    this.checkLoginStatus();
  },
  onShow() {
    this.checkLoginStatus();
  },
  methods: {
    login() {
      uni.navigateTo({
```

```
        url: "/pages/list/login/index",
      });
    },
    checkLoginStatus() {
      try {
        const storedUserInfo = uni.getStorageSync("userInfo");
        const storedIsLogin = uni.getStorageSync("isLogin");

        if (storedIsLogin === "true" && storedUserInfo) {
          this.isLogin = true;
          this.user = JSON.parse(storedUserInfo);
        } else {
          this.isLogin = false;
          this.user = null;
        }
      } catch (e) {
        console.error("读取登录状态出错:", e);
        this.isLogin = false;
        this.user = null;
      }
    },
    gotoCustomPage(code) {
      const url = this.customPageUrls[code];
      if (url) {
        // 使用 uni-app 的导航方法
        uni.navigateTo({
          url: `/pages/webview/index?url=${encodeURIComponent(url)}`,
        });
      } else {
        console.warn("未找到对应的页面 URL:", code);
      }
    },
    logout() {
      uni.showModal({
        title: "提示",
        content: "确定要退出登录吗？",
        success: (res) => {
          if (res.confirm) {
            uni.removeStorageSync("userInfo");
            uni.removeStorageSync("isLogin");

            this.isLogin = false;
            this.user = null;

            uni.showToast({
              title: "已退出登录",
              icon: "none",
```

```
                });
            }
        },
    });
  },
},
};
</script>
```

7．登录与注册

点击首页或者我的页面的授权登录，会跳转到登录页面，如果没有账号密码，可以点击底部的去注册进行注册账号。注册的账号会将数据保存在本地浏览器，登录时通过校验本地浏览器的 json 数据进行登录。除此之外，页面还添加了字段验证运行，比如说手机号必须为 11 位数，密码必须大于 6 位数。效果如下图所示：



图 21 注册页面

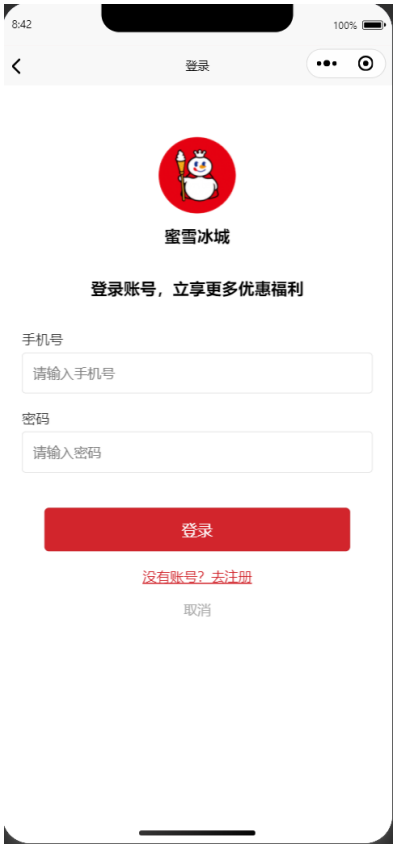


图 22 登陆页面

实现的核心代码如下：

```
<script>
export default {
  data() {
    return {
      formData: {mobile: '', password: ''},
```

```
        errors: {mobile: '', password: ''}
      };
    },
    methods: {
      validate() {
        let isValid = true;
        this.errors = {mobile: '', password: ''};

        if (!this.formData.mobile) {
          this.errors.mobile = '请输入手机号';
          isValid = false;
        } else if (!/^1\d{10}$/.test(this.formData.mobile)) {
          this.errors.mobile = '请输入正确的手机号';
          isValid = false;
        }

        if (!this.formData.password) {
          this.errors.password = '请输入密码';
          isValid = false;
        }

        return isValid;
      },

      login() {
        if (!this.validate()) return;

        try {
          const accounts = uni.getStorageSync('accounts') || '[]';
          const accountsArray = JSON.parse(accounts);
```

```
            const account = accountsArray.find(acc => acc.mobile === this.formData.mobile && acc.password
=== this.formData.password);

            if (account) {
              const userInfo = {desensitiveMobile: account.username, mobile: account.mobile};

              uni.setStorageSync("userInfo", JSON.stringify(userInfo));
              uni.setStorageSync("isLogin", "true");

              uni.showToast({title: "登录成功", icon: "success"});

              setTimeout(() => {
                uni.switchTab({url: '/pages/list/mine/index'});
              }, 1500);
            } else {
              uni.showToast({title: "用户名或密码错误", icon: "none"});
            }
          } catch (e) {
            console.error('登录失败:', e);
            uni.showToast({title: "登录失败，请重试", icon: "none"});
          }
        },

        goToRegister() { uni.navigateTo({url: '/pages/list/register/index'}); },

        goToBack() { uni.navigateBack(); }
      }
    };
</script>
    <script>
    export default {
```

```
data() {
  return {
    formData: {mobile: '', username: '', password: '', confirmPassword: ''},
    errors: {mobile: '', username: '', password: '', confirmPassword: ''}
  };
},
methods: {
  validate() {
    let isValid = true;
    this.errors = {mobile: '', username: '', password: '', confirmPassword: ''};

    if (!this.formData.mobile) {
      this.errors.mobile = '请输入手机号';
      isValid = false;
    } else if (!/^1\d{10}$/.test(this.formData.mobile)) {
      this.errors.mobile = '请输入正确的手机号';
      isValid = false;
    }

    if (!this.formData.username) {
      this.errors.username = '请输入用户名';
      isValid = false;
    } else if (this.formData.username.length < 2) {
      this.errors.username = '用户名至少需要 2 个字符';
      isValid = false;
    }

    if (!this.formData.password) {
      this.errors.password = '请输入密码';
      isValid = false;
    } else if (this.formData.password.length < 6) {
```

```
            this.errors.password = '密码至少需要 6 个字符';
            isValid = false;
        }

        if (!this.formData.confirmPassword) {
            this.errors.confirmPassword = '请确认密码';
            isValid = false;
        } else if (this.formData.confirmPassword !== this.formData.password) {
            this.errors.confirmPassword = '两次输入的密码不一致';
            isValid = false;
        }

        return isValid;
    },

    register() {
        if (!this.validate()) return;

        try {
            const accounts = uni.getStorageSync('accounts') || '[]';
            const accountsArray = JSON.parse(accounts);

            const existingAccount = accountsArray.find(account => account.username ===
this.formData.username || account.mobile === this.formData.mobile);

            if (existingAccount) {
                if (existingAccount.username === this.formData.username) {
                    this.errors.username = '该用户名已被注册';
                }
                if (existingAccount.mobile === this.formData.mobile) {
                    this.errors.mobile = '该手机号已被注册';
```

```
                }
                return;
            }

            const newAccount = {mobile: this.formData.mobile, username: this.formData.username, password: this.formData.password, registerTime: new Date().toISOString()};

            accountsArray.push(newAccount);
            uni.setStorageSync('accounts', JSON.stringify(accountsArray));

            uni.showToast({title: '注册成功', icon: 'success'});

            setTimeout(() => {
              uni.navigateTo({url: '/pages/list/login/index'});
            }, 1500);
          } catch (e) {
            console.error('注册失败:', e);
            uni.showToast({title: '注册失败，请重试', icon: 'none'});
          }
        },

        goToLogin() { uni.navigateTo({url: '/pages/list/login/index'}); },

        goToBack() { uni.navigateBack(); }
      }
    };
</script>
```

六、网站使用与维护

1、使用说明

进入首页，点击首页的授权登录，进入登陆页面，如果没有账号，可以点击底部的注册链接跳转到注册页面进行注册账号。登录成功后，会在首页和我的页面展示个人信息。根据首页的“到店自取”和“雪王外送”可以决定自取还是外卖，也可以直接进入点餐页面进行点餐。点餐时可以点击商品右侧的按钮，或者进入详情页面将商品添加到购物车，点击去结算可以进行支付。支付成功后，会跳转到订单详情页面，也可以在订单页面查看自己的历史订单记录。在我的页面可以查看自己的雪王币和优惠券，并且支持退出登录。

2、维护方案

为了蜜雪冰城点餐小程序的长久发展，需要制定一套维护方案。首先，建立完善的用户反馈机制，通过订单评价、用户反馈入口和数据分析，及时了解用户需求，优化点餐流程和功能体验。同时，确保用户数据的完整性和准确性，建立定期数据备份机制，防止信息遗漏或错乱。其次，定期更新商品信息和促销活动，借鉴其他流量较大的餐饮小程序，优化页面设计和交互体验。通过寻找合适的广告合作伙伴，制定合理的广告投放策略，提升小程序的曝光度和用户转化率。在数据安全方面，加强用户密码加密，设置访问权限控制，防止 SQL 注入攻击，并建立安全日志记录系统。最后，制定详细的维护计划，包括日常系统检查、每周数据分析、每月全面检查和季度版本更新。通过优化资源配置，控制运营成本，提升用户满意度。我们将定期评估维护效果，及时调整策略，确保小程序始终保持竞争优势，为用户提供更好的服务体验。

专用周小结

将近两周的小程序实训很快就结束了，在这段时间里，我收获了很多知识，并且熟练掌握了 vue3 语法和 uniapp 小程序设计。

在这次实训，我做的是模仿蜜雪冰城点餐系统小程序，因为结合日常生活体验，我认为类似的点餐小程序是行业趋势。在一开始，我采用老师课程中推荐到 HbuilderX 编译器创建模板和编译代码，但我发现创建的 uniapp 模板过于模板化，于是我选择了 vue 官网的 vue cli 来创建 vue3 版本的 uniapp 小程序项目。在开发过程中，HbuildeeX 的编译器内置浏览器，不方便查看网页的元素和控制台，于是我换成了 vscode 编译器结合微信开发者工具进行开发。vscode 中，拥有丰富的插件，例如代码报错提示、更强大的格式化代码、组件代码提示等，提升了开发效率。而微信开发者工具提供了不同的真实的手机模拟器来运行小程序，这样的话，就可以在开发时对灵动岛等做出合适的顶部距离判断。该项目采用纯前端方式实现，新增资源全部保存在浏览器本地。

这次项目我采用了 vue3 框架，虽然学校只教了 vue2，但我在平时自学了 vue3——因为 vue3 是前沿技术——并通过这次项目巩固了 vue3 的语法。Vue3 的语法更加自由化，更像原生 js，并且多了 hook，同时也缩小了项目的打包体积。在项目中，我采用了前沿的项目结构管理，结构分明，简明易懂，并且将许多可以封装的功能全部进行了封装，比如购物车和分类的侧边栏。

因为小程序不能直接复制、下载图片，为了获得蜜雪冰城小程序的图片资源，我特地使用了 python 爬虫，爬取了蜜雪冰城小程序的图片资源，让自己的小程序尽可能与现实中的蜜雪冰城小程序保持一致。同时在编写代码中，我遇到了很多技术问题，比如说 uniapp 的页面跳转问题等，我通过在贴吧、csdn 等专业论坛上查找信息，解决了问题

在这次实训中，我熟练掌握了小程序的开发，并且了解到了很多关于前端的新知识。关于项目的不足也有很多，比如说没有与后端和数据库的交互，功能不全等，希望在以后的学习中，

可以学习更多的代码知识，成为一名合格的程序员。

签名：

日期：2025 年 4 月 30 日

参考文献

[1] 刘培林，赵伟，申燕萍. Vue.js 跨平台开发基础教程. 机械工业出版社, 2022

[2] 刘瑞新. 网页设计与制作教程——Web 前端开发(第 6 版). 机械工业出版社, 2020

[3] 刘斌 . HTML5- 未来网络应用的核心技术研究 [J]. 自动化与仪器仪表 , 2010(4):30-33.

[4] 李晶 . NodeJS: 服务器端 JavaScript 运行环境 [J]. 程序员 , 2010(12):29-29.

评价内容	具体要求	评分	得分
1、任务完成情况	态度认真，按时完成。	20	
2、报告书规范性	报告书结构合理，文字通顺，用语准 确，符号统一，编号齐全，图表规范。 代码书写规范，且有适当注释。	40	
3、小结与参考文献撰写情况	小结写作深入具体，围绕技术问题展 开叙述；参考文献格式正确。	30	
4、创新性与功能拓展情况	除所要求完成的内容外，其他功能是否有所创新或拓展。	10	
评阅人			

时间：2025 年 4 月 30 日